# Лабораторная работа №7

## Дискретное логарифмирование в конечном поле

11 Марта 2026 - Сунь Маосин

2026

Российский университет дружбы народов, Москва, Россия

# Цель работы

# Цель работы

В данной лабораторной работе я реализовал алгоритм решения задачи дискретного логарифмирования в конечном поле $F_p$ с использованием $\rho$-метода Полларда. Задача состоит в нахождении целого числа $x$, удовлетворяющего условию $a^x \equiv b \pmod{p}$.

# Вспомогательные функции

# Вспомогательные функции

Для работы алгоритма были реализованы вспомогательные функции: расширенный алгоритм Евклида, вычисление обратного элемента по модулю и ветвящаяся функция отображения.

# Код вспомогательных функций

```python
def extended_euclid(a, b):
    """
    Extended Euclidean algorithm
    Returns (d, x, y) such that d = gcd(a, b) and a*x + b*y = d
    """
    if b == 0:
        return a, 1, 0
    else:
        d, x1, y1 = extended_euclid(b, a % b)
        x = y1
        y = x1 - (a // b) * y1
        return d, x, y
```

```python
def mod_inverse(a, n):
    """
    Compute modular inverse of a modulo n
    Returns x such that a*x == 1 (mod n)
    """
    d, x, y = extended_euclid(a, n)
    if d != 1:
        return None  # Inverse doesn't exist
    else:
        return x % n
```

```python
def f(x, a, b, p, q, G, H):
    """
    Branching function for Pollard's Rho
    Updates value and coefficients based on x mod 3
    """
    r = x % 3

    if r == 0:
        # Multiply by G
        x = (x * G) % p
        a = (a + 1) % q

    elif r == 1:
        # Multiply by H
        x = (x * H) % p
        b = (b + 1) % q

    else:  # r == 2
        # Square
        x = (x * x) % p
        a = (a * 2) % q
        b = (b * 2) % q

    return x, a, b
```

# $\rho$-метод Полларда

# $\rho$-метод Полларда

Основная функция алгоритма использует метод “быстрого и медленного указателей” для поиска коллизии. При обнаружении коллизии составляется линейное сравнение, решение которого даёт искомый дискретный логарифм.

# Код реализации

```python
def pollard_rho(G, H, P):
    '''
    Pollard's Rho algorithm for discrete logarithm
    Find x such that G^x == H (mod P)
    '''

    # Q is the order (assuming P is prime, order is P-1)
    Q = P - 1

    # Initialize
    x = (G * H) % P
    a = 1
    b = 1

    X = x
    A = a
    B = b

    # Main loop - find collision
    for i in range(1, P):
        # Update tortoise (one step)
        x, a, b = f(x, a, b, P, G, Q, H)

        # Update hare (two steps)
        X, A, B = f(X, A, B, P, G, Q, H)
        X, A, B = f(X, A, B, P, G, Q, H)

        # Check for collision
        if x == X:
            break

    # Solve for x: a - A == (B - b) * x (mod Q)
    numerator = (a - A) % Q
    denominator = (B - b) % Q

    # Compute inverse of denominator
    inv = mod_inverse(denominator, Q)

    if inv is not None:
        res = (numerator * inv) % Q
    else:
        # Try with denominator + Q
        inv2 = mod_inverse(denominator + Q, Q)
        if inv2 is not None:
            res = (numerator * inv2) % Q
        else:
            # If still no inverse, try brute force approach
            res = None
            for k in range(Q):
                if (denominator * k) % Q == numerator:
                    res = k
                    break
            if res is None:
                return None

    # Verify result
    if res is not None and pow(G, res, P) == H:
        return res
    elif res is not None:
        # Try with +Q
        res2 = (res + Q) % P
        if pow(G, res2, P) == H:
            return res2

    # If all else fails, try brute force for small P
    if P < 10000:
        for x in range(P):
            if pow(G, x, P) == H:
                return x
```

# Тестирование на примере из задания

# Тестирование на примере из задания

Для проверки корректности работы алгоритма был использован пример из задания: найти $x$ такое, что $10^x \equiv 64 \pmod{107}$.

# Код тестирования

```
def test_example():
    """Test the example from the assignment"""
    print("=" * 60)
    print("TESTING EXAMPLE FROM ASSIGNMENT")
    print("=" * 60)

    # Example from assignment
    G = 10    # generator
    H = 64    # target value
    P = 107    # prime modulus

    print(f"Find x such that {G}^x ≡ {H} (mod {P})")
    print()

    # Compute discrete logarithm
    x = pollard_rho(G, H, P)

    print(f"Result: x = {x}")
    print(f"Verification: {G}^{x} mod {P} = {pow(G, x, P)}")
    print(f"Expected: {H}")
    print(f"Success: {pow(G, x, P) == H}")

    print("\n" + "=" * 60)
```

# Результат выполнения

```
------------------------------------------------------------
TESTING EXAMPLE FROM ASSIGNMENT
============================================================
Find x such that 10^x ≡ 64 (mod 107)

Result: x = 20
Verification: 10^20 mod 107 = 64
Expected: 64
Success: True

============================================================
```

**Рис. 4:** Результат тестирования

# Пример с пользовательскими числами

# Пример с пользовательскими числами

Для демонстрации работы алгоритма на других числах был протестирован пример $5^x \equiv 3 \pmod{23}$.

# Результат выполнения

```
SOLVE DISCRETE LOGARITHM FOR TEACHER'S NUMBERS
================================================================
Enter generator (a): 5
Enter target value (b): 3
Enter prime modulus (p): 23

Finding x such that 5^x ≡ 3 (mod 23)...
----------------------------------------
Result: x = 16
Verification: 5^16 mod 23 = 3
Expected: 3
Success: True
```

# Вывод

# Вывод

В ходе лабораторной работы был успешно реализован $\rho$-метод Полларда для решения задачи дискретного логарифмирования. На примере $10^x \equiv 64 \pmod{107}$ алгоритм нашёл решение $x = 20$, что было подтверждено проверкой. Также алгоритм успешно решил пример $5^x \equiv 3 \pmod{23}$ со значением $x = 16$.